

15 | 编程接口：驾驭Headless模式，将AI能力集成到脚本与CI

Tony Bai · AI原生开发工作流实战



你好，我是 Tony Bai。

欢迎来到我们专栏进阶篇的最后一讲。在过去的几讲中，我们已经将 Claude Code 打造成了一个无所不能的“钢铁侠战甲”：它拥有持久的记忆、坚固的安全护盾，以及由自定义指令、Hooks、MCP 服务器、专家 Skills 和 Subagent 组成的、可无限扩展的能力矩阵。

但是，到目前为止，我们驾驶这套“战甲”的方式，始终是坐在这个交互式的“驾驶舱”里，通过一次次的对话来下达指令。这种模式在探索性、创造性的交互工作中非常高效。但软件工程中，还有大量需要在**无人值守的情况下、被重复、确定性地执行**的任务：

每天凌晨，自动分析前一天的服务日志，生成一份异常报告。

每次有开发者提交代码到 `main` 分支前，自动运行一个 `pre-commit` 钩子，让 AI 对代码进行一次快速的风格审查。

当 GitHub 上有新的 PR 被创建时，自动触发一个 CI/CD 流水线，让 AI 对代码变更进行一次深入的安全审计，并将报告评论到 PR 中。

这些场景，都要求 AI Agent 的能力，能够脱离交互式的终端会话，成为一个可以被其他程序调用的、标准化的“编程接口”。

这，正是我们今天要精通的终极武器——**Claude Code 的 Headless 模式（无头模式）**。

今天这一讲，我们将学习如何将强大的 Claude Code AI Agent，从一个“对话伙伴”，转变为一个可以被轻松集成到任何脚本和自动化管道中的“**可编程函数**”。你将掌握利用 `stdin/stdout`、结构化 JSON 输出以及与 CI/CD 系统集成核心技巧，完成 AI 原生开发“最后一公里”的冲刺。

什么是 Headless 模式？——让 AI 成为可编程的函数

“Headless”（无头）是一个在软件工程中非常经典的概念，它指的是一个应用程序在**没有图形用户界面（GUI）或交互式界面（CLI REPL）**的情况下运行。比如，在服务器上运行的 Chrome 浏览器，就是以 Headless 模式运行的。

Claude Code 的 Headless 模式，正是基于这一思想。它允许我们绕过那个我们熟悉的、一问一答的交互式循环，直接向 Claude Code 的“大脑”发送一个任务，然后获取一个确定的、结构化的结果，最后程序自动退出。

在这个模式下，Claude Code 的行为，从一个需要持续对话的“智能体”，蜕变为一个行为类似“纯函数”的工具：

```
f(输入) = 输出
```

输入（Input）：可以是一个直接的 Prompt，也可以是通过标准输入（stdin）管道传入的大段文本（如日志文件、代码片段）。

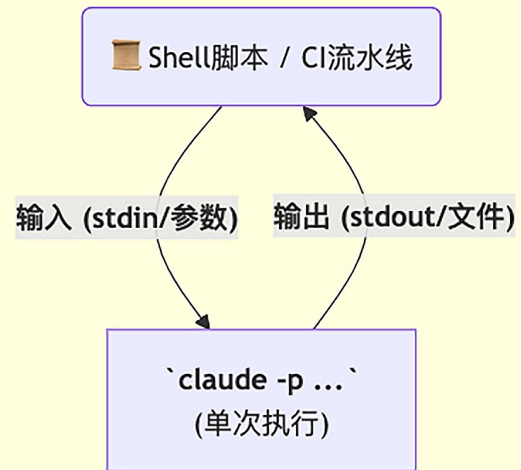
函数体（Function Body）：Claude Code 的整个 AI Agent 执行引擎，包括它加载上下文、调用工具、与大模型交互的所有复杂逻辑。

输出 (Output): 可以是纯文本的最终答案，也可以是包含完整执行过程、Token 消耗、成本等元数据的结构化 JSON。

这种转变的意义是革命性的。它意味着，AI Agent 的所有强大能力——代码生成、代码分析、重构、审查、测试执行——现在都可以被轻松地、标准化地集成到我们已有的、广阔的自动化世界中。

交互模式 vs. Headless模式

Headless模式 workflow



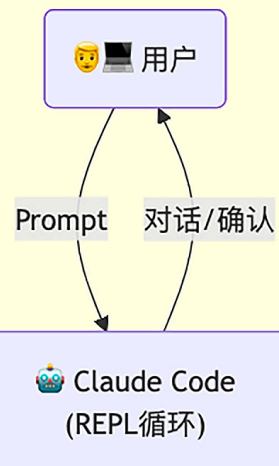
交互模式 (Interactive)
(前面14讲学习的)

心智模型: 人机对话
应用场景: 探索、创造、结对编程

Headless模式 (本讲核心)

心智模型: 函数调用
应用场景: 自动化、批处理、系统集成

交互模式 workflow



详解 `-p` 参数与输入输出流 (stdin/stdout)

开启 Headless 模式的“钥匙”，就是 `claude` 命令的一个核心参数：`-p`（或 `--print`）。一旦你使用了这个参数，Claude Code 就会明白，这是一次“非交互式”的调用。下面是在非交互 `-p` 模式下向 Claude Code 传递提示词的两种方式。

通过参数传递 Prompt

这是最简单的用法，直接将你的指令作为 `-p` 参数的值。

[复制代码](#)

```
1 # 场景：快速根据获取一个Git Commit Message建议
2 claude -p "Stage我的修改，然后生成一条符合Conventional Commit规范的Message" --allowedT
```

Claude Code 会执行这个指令，将最终生成的 Commit Message 打印到标准输出 (stdout)，然后立即退出：

 复制代码

```
1 # claude -p "Stage我的修改，然后生成一条符合Conventional Commit规范的Message" --allowe
2 Perfect! I've successfully staged all your modifications and created a commit wit
3
4 chore: add development environment configuration and update Go version
5
6 - Add Claude Code configuration and skills for Go code review
7 - Add Gemini AI development environment setup
8 - Add project constitution and documentation
9 - Update Go version from 1.21.0 to 1.25.0
10 - Include built binaries for CLI and web modes
11
12 The commit follows Conventional Commit specifications with:
13 - **Type**: `chore` (for maintenance tasks, dependency updates, and configuration
14 - **Description**: Clear and concise summary
15 - **Body**: Detailed bullet points explaining the changes
16 - **Signature**: Includes the standard Claude Code attribution
17
18 Your changes have been committed successfully and your working tree is now clean.
```

通过标准输入（stdin）管道传递上下文

当你的输入内容非常庞大时，比如一个几千行的日志文件，或者整个项目的 `tree` 结构，通过参数传递就不现实了。此时，Linux 世界最经典的“管道（Pipe）”操作就派上了用场。

Claude Code 的 Headless 模式，可以无缝地接收来自 `stdin` 的输入。

实战场景：分析 Nginx 错误日志

假设你有一个名为 `nginx-error.log` 的日志文件，你想让 AI 帮你找出其中最主要的问题。

 复制代码

```
1 # 使用cat将文件内容通过管道传递给claude
2 cat nginx-error.log | claude -p "请分析这份Nginx错误日志，总结出最主要的错误类型和可能的原
```

在这个命令中：

1. `cat nginx-error.log` 将文件的全部内容输出到 `stdout`。
2. `|` (管道符) 将 `cat` 命令的 `stdout`, 连接到了 `claude` 命令的 `stdin`。
3. `claude -p "..."` 接收到 `stdin` 的内容, 并将其作为核心上下文, 结合你的 Prompt, 发送给大模型进行分析。
4. 最终, AI 的分析报告会被打印到 `stdout`。

这种与 `stdin/stdout` 的无缝结合, 是 Headless 模式强大的根基。它意味着 Claude Code 可以像 `grep`、`awk`、`sed` 一样, 成为你强大的 Shell 工具链中, 一个具备智能的、可编程的“过滤器”或“处理器”。

结构化输出：精通 `--output-format`, 获取 JSON 结果

如果说 `stdin` 解决了输入的标准化, 那么 `--output-format` 参数则解决了输出的标准化。这对于将 AI 能力集成到自动化脚本中至关重要。

当你需要对 AI Agent 的输出进行编程化处理时, 比如你想判断任务是否成功、获取执行成本、或者解析 AI 调用的某个工具的结果, 纯文本的输出就显得不够用了。你需要的是机器友好的结构化数据。

Claude Code 的 `--output-format` 参数提供了三种选择: `text` (默认)、`json` 和 `stream-json`。

json 格式：获取包含完整元数据的“执行报告”

当你使用 `--output-format json` 时, Claude Code 的输出将不再是简单的最终答案, 而是一个包含了整个执行过程的、完整的 JSON 对象。

实战场景：记录一次代码审查的成本

 复制代码

```
1 claude -p "使用go-code-security-reviewer subagent 审查@internal/converter/converte
```

打开 `review_report.json` 文件，你会看到一个类似这样的结构：

 复制代码

```
1 {
2   "type": "result",
3   "subtype": "success",
4   "is_error": false,
5   "duration_ms": 28005,
6   "duration_api_ms": 40230,
7   "num_turns": 4,
8   "result": "The go-code-security-reviewer subagent has completed a comprehensive
9   "session_id": "1384919c-ea11-43dc-8eb9-26444cae76",
10  "total_cost_usd": 0.18573870000000003,
11  "usage": {
12    "input_tokens": 16214,
13    "cache_creation_input_tokens": 0,
14    "cache_read_input_tokens": 14912,
15    "output_tokens": 476,
16    "server_tool_use": {
17      "web_search_requests": 0
18    },
19    "service_tier": "standard",
20    "cache_creation": {
21      "ephemeral_1h_input_tokens": 0,
22      "ephemeral_5m_input_tokens": 0
23    }
24  },
25  "modelUsage": {
26    "glm-4.6": {
27      "inputTokens": 33613,
28      "outputTokens": 3524,
29      "cacheReadInputTokens": 79456,
30      "cacheCreationInputTokens": 0,
31      "webSearchRequests": 0,
32      "costUSD": 0.17753580000000002,
33      "contextWindow": 200000
34    },
35    "glm-4.5-air": {
36      "inputTokens": 1854,
37      "outputTokens": 167,
38      "cacheReadInputTokens": 453,
39      "cacheCreationInputTokens": 0,
40      "webSearchRequests": 0,
41      "costUSD": 0.008202899999999999,
42      "contextWindow": 200000
43    }
44  },
```

```
45     "permission_denials": [],
46     "uuid": "c6c642e3-2d9e-4aed-98fe-d290c01f0759"
47 }
```

这个 JSON 就是一个**完整的、可审计的“执行日志”**。在自动化脚本中，你可以轻松地使用 `jq` 等工具来解析它：

想获取最终的审查结果？ `jq '.result'`

想知道这次任务花了多少钱？ `jq '.total_cost_usd'`

想知道任务是否成功？ `jq '.subtype'`

`json` 格式是你将 AI 集成到严肃的、需要进行监控和审计的自动化流程中的不二之选。

`stream-json` 格式：实时获取 AI 的“心跳”

`json` 格式会在所有任务完成后，一次性地输出一个完整的 JSON 数组。但如果任务需要很长时间，你可能希望实时地看到 AI 的进展。

`--output-format stream-json` 就是为此而生的。它会在 AI 执行的每一步，都立即向 `stdout` 输出一个独立的 JSON 对象，每行一个（`jsonl` 格式）。

 复制代码

```
1 # 实时观察AI的思考过程
2 claude -p "使用go-code-security-reviewer subagent 审查@internal/converter/converte
```

注：`stream-json` 输出格式必须与 `-verbose` 一起使用。

你的终端会像看直播一样，实时地滚动输出 AI 的每一个思考（`assistant` 消息）、每一次工具调用（`tool_use`）、每一次工具结果（`tool_result`）...

这种格式非常适合：

构建交互式应用：你可以编写一个程序，实时解析 `stream-json` 的输出，并将其渲染成一个更漂亮的、带实时状态更新的 UI。

需要实时反馈的 CI/CD 流程：在 GitHub Actions 的日志中，你可以实时地看到 AI 正在进行到哪一步，而不是面对一个长时间的静默等待。

实战一：编写 Shell 脚本，调用 AI 能力分析日志文件

现在，让我们将所学知识融会贯通，编写一个实用的 Shell 脚本 `analyze_log.sh`。它的功能是接收一个日志文件名作为参数，调用 Claude Code 对日志进行分析，并将分析结果（仅纯文本）保存到一个报告文件中。

```
analyze_log.sh
```



```
1  #!/bin/bash
2
3  # 检查参数
4  if [ -z "$1" ]; then
5      echo "Usage: $0 <log_file_path>"
6      exit 1
7  fi
8
9  LOG_FILE=$1
10 REPORT_FILE="${LOG_FILE}.report.txt"
11 PROMPT="你是一个资深的SRE工程师。请深入分析这份日志文件，找出其中的错误（Error）
和警告（Warning），对它们进行分类和计数，并总结出最可能导致这些问题的根本原因。以Ma
rkdown格式输出你的报告。"
12
13 echo "Analyzing log file: $LOG_FILE..."
14 |
15 # 使用管道将日志文件内容传递给Claude Code的Headless模式
16 # 使用 --output-format json 来获取结构化结果
17 # 使用 jq 提取最终的纯文本报告
18 # 使用 > 将输出重定向到报告文件
19 cat "$LOG_FILE" | claude -p "$PROMPT" --output-format json | jq -r '.r
esult' > "$REPORT_FILE"
20
21 # 检查命令是否成功执行
22 if [ $? -eq 0 ]; then
23     echo "✅ Analysis complete. Report saved to: $REPORT_FILE"
24     echo "--- Report Preview ---"
25     head -n 10 "$REPORT_FILE"
26 else
27     echo "❌ Analysis failed. Please check your Claude Code setup and API
key."
28     # 尝试打印一些错误信息
29     cat "$LOG_FILE" | claude -p "$PROMPT" --output-format json > /dev/nu
ll
30     exit 1
31 fi
```

这个脚本完美地演示了 Headless 模式的威力：

1. 它将一个复杂的 AI 调用，封装成了一个**可复用的命令行工具**。
2. 它通过管道，优雅地处理了任意大小的输入。
3. 它通过 `--output-format json` 和 `jq` 的组合，精确地提取出了它需要的最终结果，并忽略了所有中间过程的元数据。
4. 它通过输出重定向，将 AI 的产出无缝地集成到了我们的文件系统中。

实战二：在 GitHub Actions 中集成 Claude Code，实现自动化流程

Headless 模式最激动人心的应用场景，无疑是与 CI/CD 系统的集成。让我们来设计一个 GitHub Actions 工作流，它会在每次有新的 PR 被创建时，自动触发 Claude Code，对代码变更进行一次快速的审查。

`.github/workflows/claude_pr_review.yml`

 复制代码

```
1 name: AI Code Review
2
3 on:
4   pull_request:
5     types: [opened, synchronize]
6
7 jobs:
8   ai-review:
9     runs-on: ubuntu-latest
10    permissions:
11      contents: read # 需要读取代码
12
13    steps:
14      - name: Checkout code
15        uses: actions/checkout@v4
16        with:
17          # 获取足够的历史记录以便进行diff
18          fetch-depth: 0
19
20      - name: Setup Go
21        uses: actions/setup-go@v5
22        with:
```

```
23         go-version: '1.25.0'
24
25     - name: Install Claude Code
26       run: npm install -g @anthropic-ai/claude-code
27
28     - name: Configure Claude Code with Zhipu AI
29       env:
30         ZHIPU_API_KEY: ${ secrets.ZHIPU_API_KEY }
31       run: |
32         mkdir -p ~/.claude
33         echo '{
34           "env": {
35             "ANTHROPIC_BASE_URL": "https://open.bigmodel.cn/api/anthropic",
36             "ANTHROPIC_API_KEY": "'$ZHIPU_API_KEY'"
37           }
38         }' > ~/.claude/settings.json
```

```

1      - name: Run AI Review in Headless Mode
2        id: review
3        run: |
4          # 准备Prompt
5          PROMPT="你是一个资深的Go代码审查员。请分析本次Pull Request中的代码变
更，重点关注潜在的bug、不符合Go语言惯用风格的地方、以及性能问题。以Markdown列表的
形式，给出你的审查建议。如果没有任何问题，请回复'代码质量很好，没有发现明显问题。'"
6
7          # 获取diff内容并通过管道传递
8          # ${ github.base_ref } 指向目标分支，${ github.head_ref }
指向源分支
9          git diff origin/${ github.base_ref }...origin/${ github.h
ead_ref } | claude -p "$PROMPT" --output-format json > result.json
10
11         # 检查执行结果
12         if [ $? -ne 0 ]; then
13           echo "AI review command failed."
14           exit 1
15         fi
16
17         # 提取审查报告，并将其设置为Action的输出
18         REPORT=$(jq -r '.result' result.json)
19         # 将多行字符串转换为GitHub Actions输出需要的格式
20         echo "report_content<<EOF" >> $GITHUB_OUTPUT
21         echo "$REPORT" >> $GITHUB_OUTPUT
22         echo "EOF" >> $GITHUB_OUTPUT
23
24     - name: Post Review Comment to PR
25       uses: actions/github-script@v7
26       with:
27         script: |
28           const report = `${ steps.review.outputs.report_content }
`;
29
30           // 只有当报告不是默认的“无问题”消息时，才发表评论
31           if (report && !report.includes("没有发现明显问题")) {
32             github.rest.issues.createComment({
33               issue_number: context.issue.number,
34               owner: context.repo.owner,
35               repo: context.repo.repo,
36               body: `### 🤖 AI Code Review\n\n${report}`

```

```
37         } else {  
38             console.log("AI review found no issues, skipping comment.");  
39         }
```

这个工作流的精妙之处在于：

1. **完全自动化**：它在无人值守的 CI 环境中，完整地执行了一个“获取上下文 -> 调用 AI -> 处理结果 -> 反馈”的闭环。
2. **上下文精准**：它通过 `git diff` 和管道，精确地将本次 PR 的增量代码作为上下文喂给了 AI。
3. **结果可编程**：它利用 `--output-format json` 和 `jq`，将 AI 的审查报告提取出来，并传递给后续的 Step。
4. **智能交互**：最后一个 Step 利用 `actions/github-script`，将 AI 的报告评论回当前的 Pull Request，实现了 AI 与开发者的异步协作。

这，就是 AI 原生开发工作流在自动化管道中的最终形态。

本讲小结

今天，我们一起解锁了将 AI Agent 从“交互式伙伴”变为“自动化引擎”的终极密码——Headless 模式，这标志着我们已经具备了将 AI 能力融入任何复杂工程场景的底层能力。

首先，我们理解了 **Headless 模式** 的核心理念，即像调用一个“可编程函数”一样，非交互式地执行 AI 任务。接着，我们深入学习了开启 Headless 模式的钥匙——`-p` 参数，并掌握了如何通过标准输入输出流（stdin/stdout）来处理大规模的上下文输入。

然后，我们学习了 `--output-format` 参数，特别是强大的 `json` 和 `stream-json` 格式，学会了如何获取用于编程和审计的结构化结果。最后，我们通过两个极具价值的实战——**分析日志的 Shell 脚本** 和 **自动化 PR 审查的 GitHub Actions**——将所有理论知识融会贯通，完成了 AI 原生开发“最后一公里”的冲刺。

掌握了 Headless 模式，你就拥有了一把“万能钥匙”，可以打开 AI 与你现有工具链、脚本和 CI/CD 系统之间的所有大门。你的想象力，将成为 AI 能力边界的唯一限制。至此，我们已经学完了 AI 原生开发所需要的 Claude Code 所有“武功秘籍”。从下一讲开始，我们将正式进入本专栏最激动人心的模块四：实战篇。

我们将不再学习新的孤立知识点，而是将前面所有学到的——规范驱动开发、上下文艺术、安全基石、能力扩展——全部整合起来，在一个从零到一的 Go 项目中，进行一次酣畅淋漓的“全流程演练”。我们的第一步，将从一个宏大的视角开始：顶层设计——构建你的 AI 原生开发“驾驶舱”。我们将学习顶级团队是如何设计他们的 Claude Code 协作框架的。

思考题

我们今天的 GitHub Actions 实战，实现了一个“AI Code Reviewer”。但它还比较初级。请你打开思路，思考一下，如何利用我们今天学到的 Headless 模式和 json 输出格式，对这个 CI 工作流进行**升级**？

例如，我们能否解析 AI 审查报告的**结构**（比如 AI 是否标记了“高优先级”问题），并根据此来决定这个 CI 流水线是**通过（Pass）**还是**失败（Fail）**？如果可以，你会在 `github-script` 步骤中如何实现这个逻辑？

欢迎在评论区分享你的设计方案！这会是一个非常棒的、关于 AI 与 DevOps 深度融合的思维练习。

AI智能总结

1. Headless模式是指在软件工程中应用程序在没有图形用户界面或交互式界面的情况下运行，允许AI Agent的能力被轻松地、标准化地集成到自动化世界中。
2. Headless模式的意义在于将AI Agent的所有强大能力轻松地、标准化地集成到自动化世界中，提高工作效率和准确性。
3. Headless模式的应用场景包括自动分析服务日志、代码审查、CI/CD流水线触发等，要求AI Agent的能力能够脱离交互式的终端会话，成为一个可以被其他程序调用的、标准化的“编程接口”。
4. 在GitHub Actions中集成Claude Code，实现自动化流程，可以实现完全自动化的CI环境中的“获取上下文 -> 调用AI -> 处理结果 -> 反馈”的闭环。
5. GitHub Actions工作流通过`git diff`和管道，精确地将本次PR的增量代码作为上下文喂给了AI，实现了上下文精准。
6. GitHub Actions工作流利用`--output-format json`和`jq`，将AI的审查报告提取出来，并传递给后续的Step，实现了结果可编程。
7. GitHub Actions工作流利用`actions/github-script`，将AI的报告评论回当前的Pull Request，实现了AI与

- 开发者的异步协作。
8. Headless模式的核心技巧包括利用`stdin/stdout`、结构化JSON输出以及与CI/CD系统集成，完成AI原生开发“最后一公里”的冲刺。
9. 掌握了Headless模式，可以打开AI与现有工具链、脚本和CI/CD系统之间的所有大门，拥有了一把“万能钥匙”。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (10)

最新 精选



AKA三皮
2025-12-22 来自江苏

老师，在文中提到的 实战一：编写 Shell 脚本，调用 AI 能力分析日志文件，里面并没有跟 Claude 的交互，这条命令是不是有问题：`cat "PROMPT" --output-format json | jq -r '.result' > "$REPORT_FILE"`

作者回复: markdown转义问题，已经让编辑老师安排修正了。感谢指出。

共 2 条评论 > 4



wiekern
2025-12-23 来自广东

Claude Code 客户端 + Claude Opus 4.5模型 的方式 和 现在的 Cursor/Windsurf 选择 Claude Opus4.5模型 的根本区别是什么？

作者回复: 一般认为：Cursor 是给你配了个顶级副驾驶，Claude Code 是给你雇了个能干脏活累活的初级员工。模型一样（大脑一样），但身体和手脚不同:）。cc不仅能写代码，还能自己跑测试、修 Bug、查日志、发 PR, 你可以把它集成到 CI/CD 流水线里，让它半夜自动修 Bug。它的工具调用能力（Tool Use）和系统权限远高于 魔改的IDE或IDE 插件。

共 2 条评论 > 3



AKA三皮
2025-12-22 来自江苏

最近看到Claude官方的一个讨论，大概意思是建议大家不要去卷agent了，应该去卷skills。如果Claude code 是一个通用的agent，并且可以通过headless来实现编程，那么是否意味着，我可以把Claude看作一个agent core，基于它去打造属于自己/公司的agent。可以有ui，可以使用业务系统的认证授权体系，等等～

作者回复: 我觉得anthropic也在claude code的使用上处于探索阶段。你说的没错。目前一个方向就是将claude code视为一个agent的runtime，或者你说的agent core，后续基于该runtime/core，打造和搭建自己的agent。

共 2 条评论 >

👍 3



一只豆

2025-12-25 来自广东

codex 和 gemini cli 是否也具备 headless 模式呢？

作者回复: 我记得gemini cli肯定是支持的。



👍 2



毛毛

2026-01-07 来自广东

headless模式会不会消耗token较多 特别是后台服务 还有怎么检验 headless模式结果的准确性

作者回复: Headless 模式通常是一次性任务（如“审查 PR”），没有多轮对话的上下文累积，理论上，单次任务的 Token 消耗比人机对话要少。

至于准确性校验，这个要视具体任务类型来判断了。可以用“副作用校验”的策略，比如如果是写代码的任务，可以依靠测试（比如go test）来验真。如果是审查的任务，那现阶段多数团队可能还得依靠人工抽检。即使是自动化流程，在高风险操作前（如合并 PR），依然建议保留一个人工 Approve 的环节。



👍 1



孙鑫

2026-01-21 来自上海

有没有这种可能？就是说我把claude code本身当做一个业务框架引擎放在服务器上，把对应的业务写好，然后通过headless的方式做成标准化的API接口，然后对接给前端服务，有实现

的可行性吗？替代传统的后端开发架构？

作者回复: 技术上可行（可行性），但经济和性能上目前不可行（合理性）。

你提到的正是 LLM OS 的雏形。Claude Code 有工具调用能力（查库、调接口）和推理能力。你可以把它看作一个“万能后端”，前端发需求（自然语言），它实时生成 SQL、执行逻辑并返回结果。

但为什么现在不能替代传统后端？

延迟太高，传统 API 响应是毫秒级，Claude Code 思考+执行是秒级甚至分钟级。用户等不起。成本太贵，每次 API 调用都要消耗海量 Token，成本是传统计算的成千上万倍。

另外它也不可控，它的逻辑是概率性的，同一套输入可能产生不同输出，不适合做严肃的业务逻辑层。

因此，目前headless模式适合做离线任务（如报表生成、日志分析），暂不适合做在线业务（如订单支付）。

共 2 条评论 >



Geek_90adaf

2026-01-21 来自浙江

老师 请教下，使用claude code来审核代码 和直接调用api对比，有什么优势吗，在我看来直接调用api会更方便

作者回复: 我理解你所谓的直接调用api，是直接使用大模型api发起review请求，并通过response获得review结果。基于这个前提，我们来看看两种方式的区别。

Claude Code 的优势在于“环境感知”和“工具闭环”。

直接调 API，你只是在和一个文本生成器对话。你得自己把代码复制粘贴发给它。它发现了问题，也只能告诉你“这里错了”，不能帮你修。你还得自己去验证它的建议对不对。

Claude Code 是“活”在代码库里的 Agent。它能主动感知，即它可以自己 git diff、grep、read 文件，甚至顺藤摸瓜去查引用的定义(也可以通过 lsp 语言服务器工具)，看到的上下文更完整。它还可以验证闭环，它可以自己跑 linter 或 test 来验证代码是否有问题，甚至可以直接提交修复代码。

API 只能“看”，Claude Code 能“看、查、验、改”。



Geek_bc2516

2026-01-02 来自浙江

太厉害了，要是我会写shell脚本的话，配上这个确实能做很多事情

作者回复: 是的。

shell脚本可以结合ai来实现，而且幻觉率较低。

共 2 条评论 >



子衿

2025-12-23 来自浙江

来啦

作者回复: [手动抱拳]



Geek_2a62e6

2025-12-22 来自广东

在第一个脚本中 `cat "<!--$$MATH_0$$-->PROMPT"` 这个是什么意思，这里应该是`claude $PROMPT` 是不是呢

作者回复: 应该是编辑器转义的问题，我让编辑老师看一下。

